

Online Pharmacy Platform

Low-Level Design

Service-Level Class Diagrams & Component Detail

This document provides a detailed low-level design for each microservice of the Online Pharmacy Platform, including class hierarchies, service responsibilities, data models, event publishers, security filters, and inter-service clients.

1. Identity Service

1.1 Responsibilities

- User registration (with OTP email verification) and JWT-based authentication.
- OTP generation and validation for signup, login MFA, and password reset flows.
- JWT blacklisting on logout using Redis with TTL equal to token expiry.
- Address CRUD with default-address management per user.
- In-app notification CRUD (create, list, mark-as-read, delete).
- Publishes UserRegisteredEvent and UserLoggedInEvent to RabbitMQ.

1.2 Class Diagram

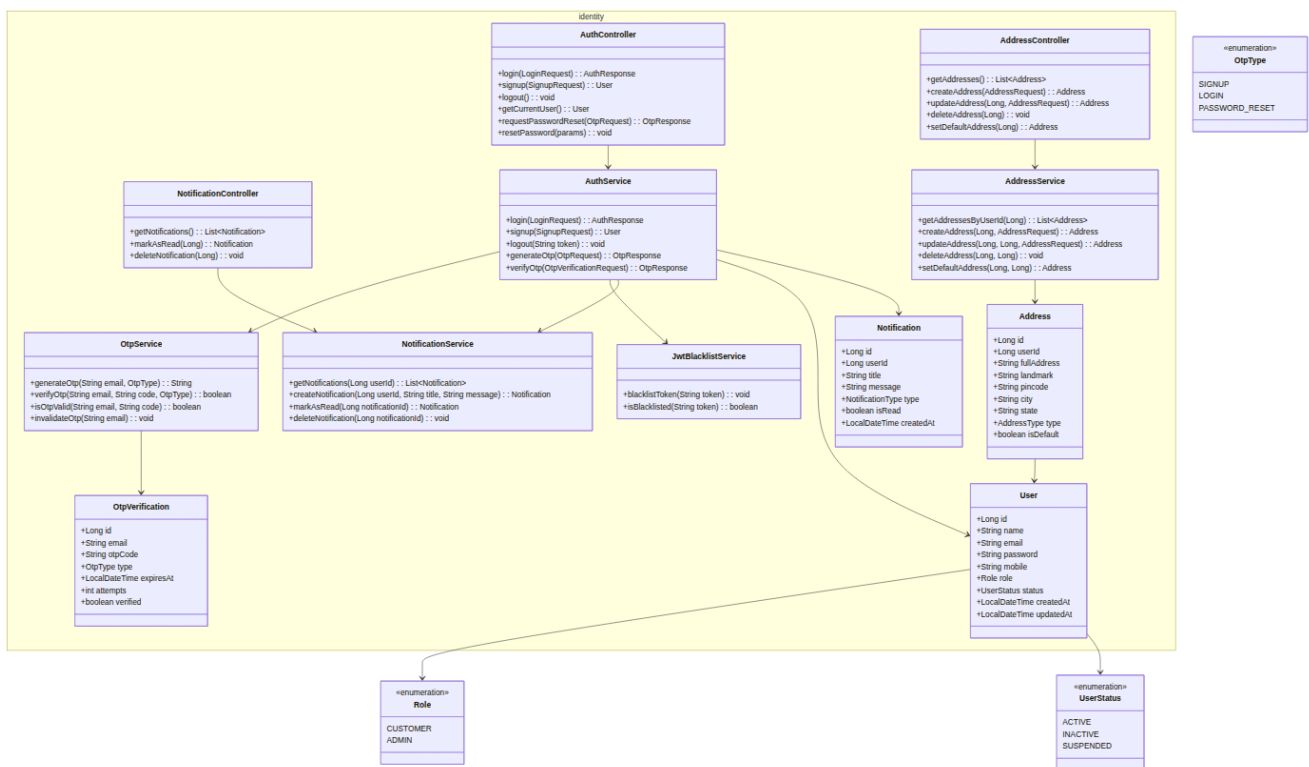


Figure 1 — Identity Service Class Diagram

1.3 Key Classes

AuthController — REST entry point for `/auth/**` endpoints. Delegates to **AuthService**.

AuthService — Orchestrates login, signup, OTP verification, logout. Publishes events.

OtpService — Generates time-bound OTP codes, validates attempts, invalidates after use.

JwtBlacklistService — Wraps Redis to blacklist JWTs and check blacklist status on each request.

AddressService — Manages addresses scoped to a `userId`; enforces single default address.

NotificationService — CRUD for in-app notifications stored in `auth_db.notifications`.

User / Address / Notification / OtpVerification — JPA entities mapped to `auth_db` tables.

2. Catalog Service

2.1 Responsibilities

- Medicine CRUD with full-text search, category filtering, and paginated listing.
- Category management (hierarchical, with image support).
- Inventory batch management: track quantity, expiry, and low-stock thresholds.
- Prescription image upload (local file storage), review (approve/reject) with events.
- Handles stock reservation and release requests from the Orders Service.
- Publishes InventoryReservedEvent, InventoryReleasedEvent, PrescriptionApprovedEvent, PrescriptionRejectedEvent.

2.2 Class Diagram

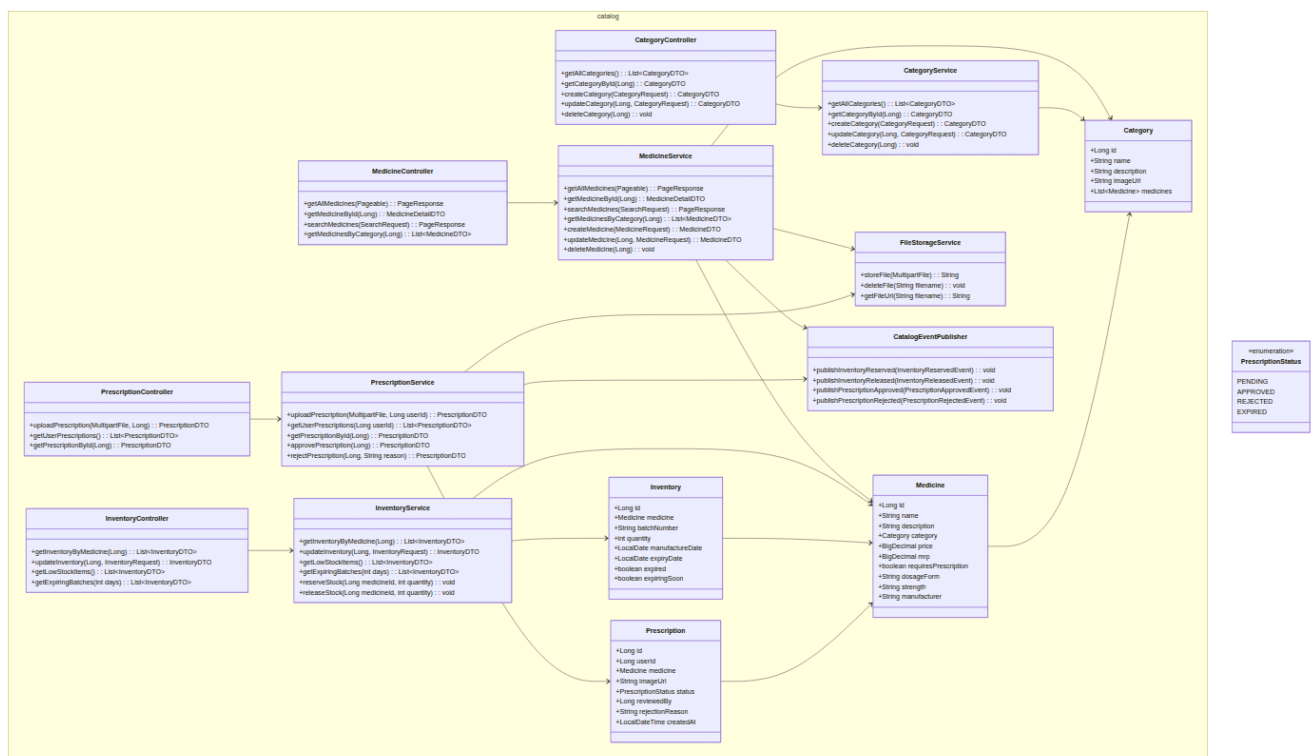


Figure 2 — Catalog Service Class Diagram

2.3 Key Classes

MedicineService — Full medicine lifecycle; delegates file operations to FileStorageService.

InventoryService — Manages batch quantities; exposes reserveStock / releaseStock for inter-service calls.

PrescriptionService — Uploads prescription images, tracks status transitions, triggers events on review.

CatalogEventPublisher — Wraps RabbitMQ template to publish domain events with routing keys.

FileStorageService — Stores uploaded prescription images to disk; returns public URL.

Medicine / Category / Inventory / Prescription — JPA entities for catalog_db.

3. Orders Service

3.1 Responsibilities

- Cart lifecycle: add/update/remove items, clear cart, one-cart-per-user enforcement.
- Checkout: validate cart, call Catalog to reserve inventory, create Order (PAYMENT_PENDING).
- Payment processing for COD, Card, UPI, Wallet; handles success and failure paths.
- Order cancellation with inventory release and optional refund initiation.
- Return/refund workflow initiation.
- Publishes OrderPlacedEvent and OrderCancelledEvent to RabbitMQ.

3.2 Class Diagram

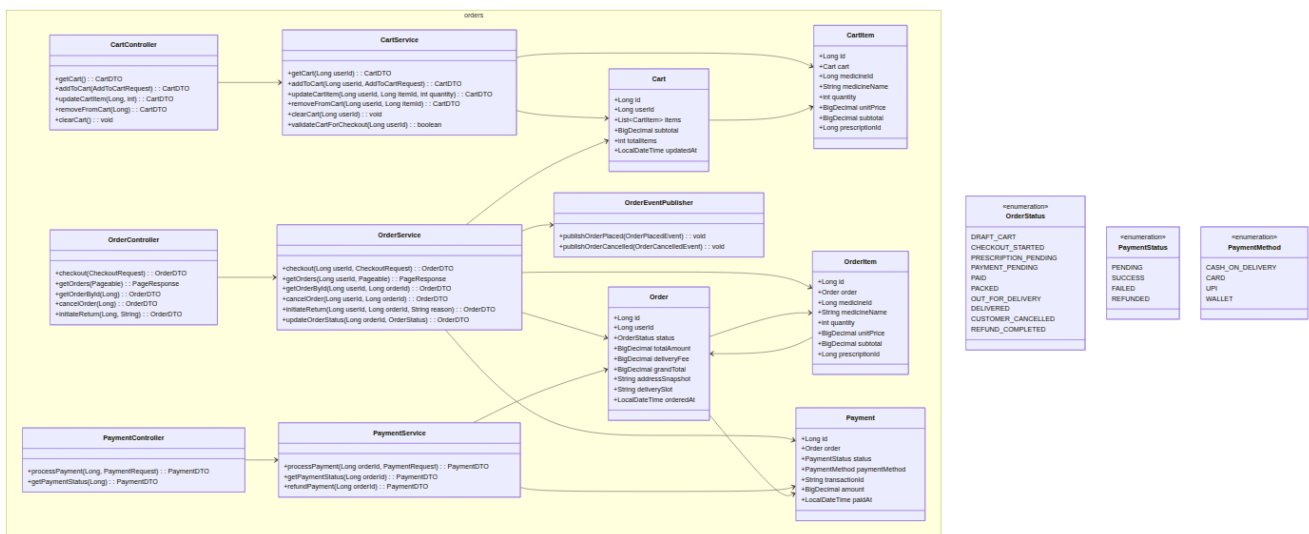


Figure 3 — Orders Service Class Diagram

3.3 Key Classes

CartService — Manages cart state in order_db; validates cart contents before checkout.

OrderService — Orchestrates checkout, calls MedicineClient for inventory, persists Order/OrderItem.

PaymentService — Processes payment records; triggers inventory release on failure.

OrderEventPublisher — Publishes order domain events to RabbitMQ.

Order / OrderItem / Cart / CartItem / Payment — JPA entities for order_db.

OrderStatus (enum) — 16-state lifecycle from DRAFT_CART through REFUND_COMPLETED.

4. Admin Service

4.1 Responsibilities

- Dashboard KPI aggregation: total/pending/completed orders, revenue, prescription counts.
- Low-stock and expiring-batch monitoring (delegates to CatalogClient).
- Prescription approval and rejection workflow.
- Order status management for admin-initiated updates.
- Acts as a BFF (Backend-For-Frontend) aggregating data from Catalog and Orders via Feign clients.
- Secured by JWT header propagation — no own auth; relies on Gateway-forwarded user principal.

4.2 Class Diagram

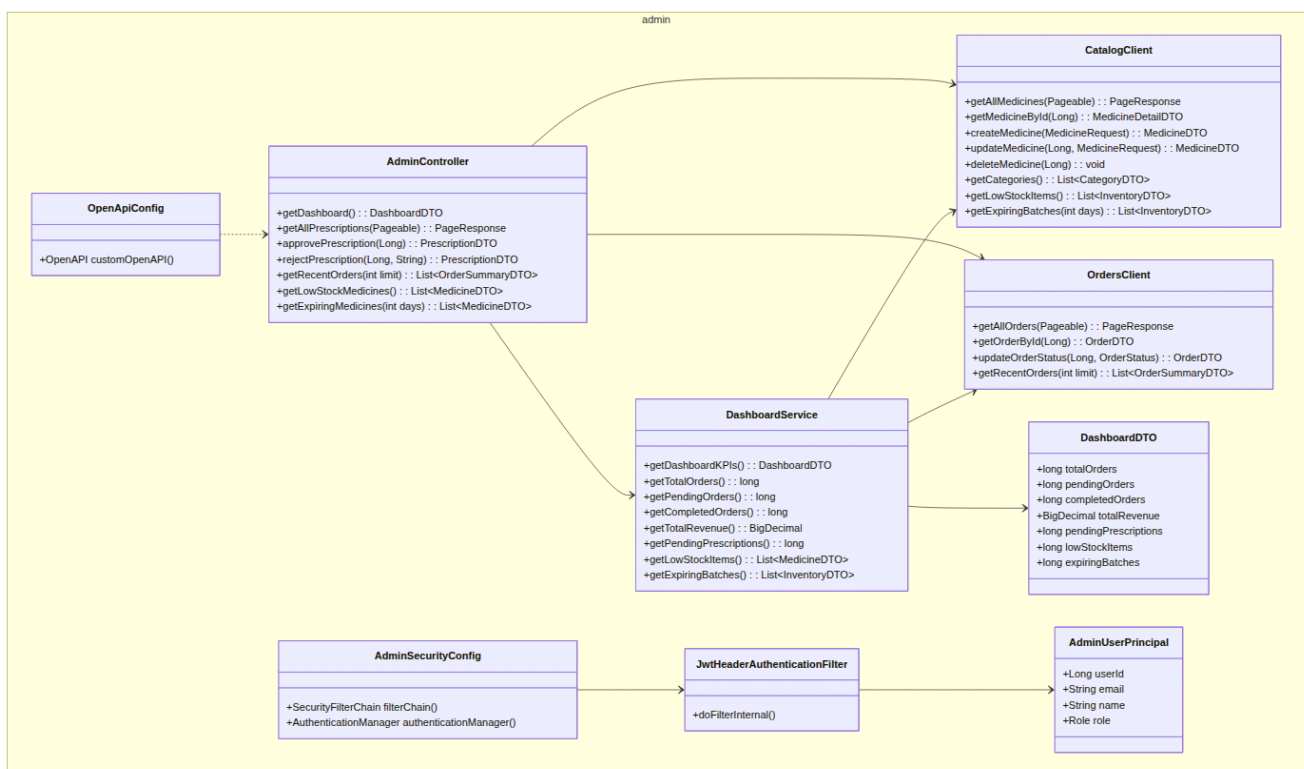


Figure 4 — Admin Service Class Diagram

4.3 Key Classes

AdminController — Single REST controller exposing dashboard and management endpoints.

DashboardService — Aggregates KPIs by calling CatalogClient and OrdersClient; returns DashboardDTO.

CatalogClient — Feign client wrapping Catalog Service REST API for medicine and inventory operations.

OrdersClient — Feign client wrapping Orders Service REST API for order listing and status updates.

JwtHeaderAuthenticationFilter — Reads Gateway-forwarded user headers; populates SecurityContext.

5. Notifications Service

5.1 Responsibilities

- Consumes all domain events from RabbitMQ (user, order, prescription, inventory).
- Dispatches transactional emails via SMTP (Mailpit in dev, real provider in production).
- Persists in-app notifications; exposes unread count and history APIs.
- Fetches user email via IdentityClient (Feign) when not available in event payload.
- Stateless processing — no own database for business data; only notification_log table.

5.2 Class Diagram

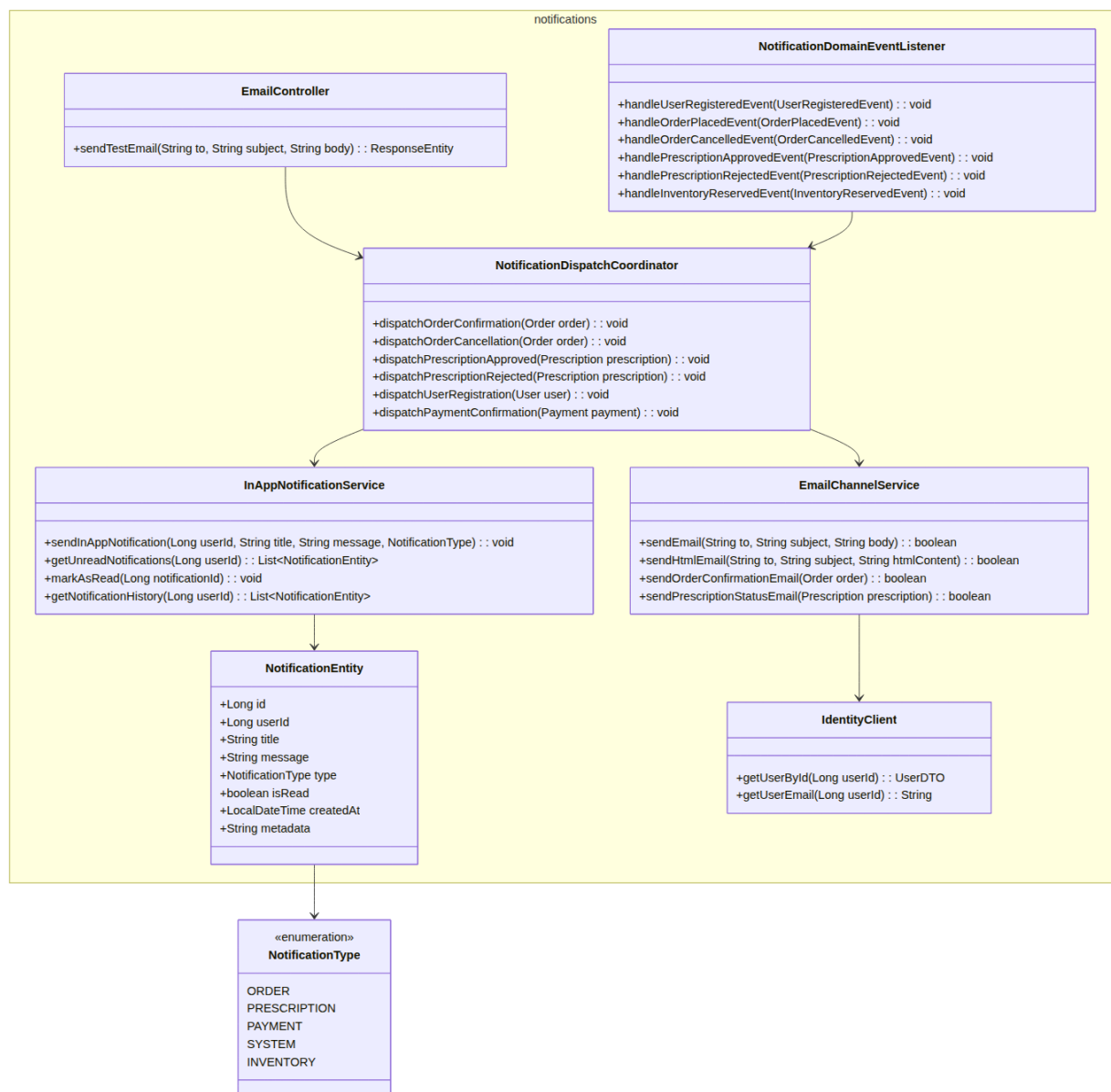


Figure 5 — Notifications Service Class Diagram

5.3 Key Classes

NotificationDomainEventListener — RabbitMQ `@RabbitListener`; one handler per event type.

NotificationDispatchCoordinator — Decides which channels (email, in-app) to invoke for each event.

EmailChannelService — JavaMailSender wrapper; constructs HTML templates and sends emails.

InAppNotificationService — Persists and retrieves NotificationEntity records.

IdentityClient — Feign client to resolve user email from Identity Service.